Performances of Protein Structure Alignment Algorithms under the MapReduce Framework

Chen-En Hsien Guan-Jie Hua Yaw-Ling Lin
Department of Computer Science and Information Engineering
Providence University
Taichung, Taiwan
seanm7739@gmail.com
gt758215@gmail.com
yllin@pu.edu.tw

Che-Lun Hung
Department of Computer Science and Communication Engineering
Providence University
Taichung, Taiwan
clhung@pu.edu.tw

Abstract

Protein structures are essential for correct biological functions because similar structures between proteins allow molecular recognition. Identifying similar structures between proteins provide the opportunity to recognize homology that is undetectable by sequence comparison. Thus comparison and alignment of protein structures represents a powerful means of discovering functions, yielding direct insight into the molecular mechanisms. Currently, there are several techniques available in attempting to find the optimal alignment of shared structural motifs between two proteins.

This paper adopts approach using the MapReduce paradigm to parallelize tools and manage their execution. The experiment shows that the map/reduce-paralleled from the original sequential combinatorial algorithm performs well on the real-world data obtained in from the PDB data set; the computation efficiency can be effectively improved proportional to the number of processors being used.

Keywords: Cloud Computing; Map/Reduce; Protein Structures

1 Introduction

Protein structures play critical roles in vital biological functions [1]. With more than 40,000 protein structures determined by the advances in X-ray crystallography and NMR spectroscopy to date, molecular biologists these days proceed in the direction of analyzing and classifying these protein structures in order to discover the structural relationships with protein functions [2].

Detection of proteins with a similar fold can suggest a common ancestor, and often a similar function [3]. Comparison of 3D structures makes it possible to establish distant relationships, even between protein families distinct in terms of sequence comparison alone. This is why structural alignment of proteins increases our understanding of more distant evolutionary relationships [4] [5]. The link between structural classification and sequence families enables us to study functions of various folds, or whole proteins [6].

There have been several methods proposed to compare protein structures and measure the degree of structural similarity based on alignment of secondary structure elements as well as alignment of intra and inter-molecular atomic distances. The basic ideas are rapid identification of pair alignments of secondary structure elements, clustering them into groups, and scoring the best substructure alignment. For examples, the VAST system [7] is based on continuous distribution of domains in the fold space. The FSSP/DALI system [8] provides two levels of description - a coarsegrained one and one with a fine-grained resolution. The method, CATH, provides the complete PDB fold classification by domains and links to other sources of information. The two methods, CE and LGscore2 [9] focus on the local geometry rather than global features such as orientation of secondary structures and overall

topology (as in the case of VAST or DALI). VAST has been used to compare all known PDB domains to each other. The results of this computation are included in NCBI's Molecular Modelling Database at http://www.ncbi.nlm.nih.gov/Structure/VAST/vast.html.

Note that there must be an atom-pairing scheme before one can do the structure alignment computation. The first atom of the first selection is compared to the first atom of the second selection, fifth to fifth, and so on. Incorporating with ideas of bipartite matching and 3-parameter isometric transformation, Lin et al. [10], [11] proposed methods of using parametric searching strategies with adaptive controls, and demonstrated that more accurate and similar protein structure pairings are possible comparing to previous known results like VAST [7] or CE [9].

One of the crucial steps of these algorithms is finding a good isometric transformation, which leads to the best atom-pairing alignment between two proteins. In this paper, we propose algorithms of for efficiently locating more suitable isometric transformations of one structure and aligning it to the other structure. Based upon the periodical property of the parametric settings, we propose parametric searching strategies by approximations with power series and trigonometric series. We show the eRectiveness of the proposed parametric searching strategies by a set of experiments, which leads to better alignments of structure pairing in general.

Hadoop [12] is a software framework intended to support data-intensive distributed applications. It is able to process petabytes of data with thousands of nodes. Hadoop supports MapReduce programming model [13] for writing applications that process large data set in parallel on Cloud Computing environment. The advantage of MapReduce is that it allows for distributed computing of the map and reduction operations. Each map operation is independent of the others and all maps can perform the tasks in parallel. In practice, the number of the map is limited by the data source and/or the number of CPUs near that data. Similarly, a set of reducers can perform the reduce operations. All outputs of the map operations which share the same key are presented to the same reducer, at the same time. In addition, one of the important benefits to use Hadoop to develop the applications is due to its high degree of fault tolerance. Even when running jobs on a large cluster where individual nodes or network components may experience high rates of failure, Hadoop can guide jobs toward a successful completion. Many applications of bioinformatics are often computationconsuming; sometimes it needs weeks or months to complete the jobs. The traditional parallel models, such as MPI, OpenMP and Multi-thread, are not suitable to such applications, where a fault occurred in some nodes leads the entire application into total failure. In these situations, the Hadoop platform are considered as a much better solution for these real-world applications. Recently, Hadoop has been applied in various domains in bioinformatics [14], [15].

2 Method and Materials

A. Protein structure Analysis

The main idea of our local refinement algorithm for finding a suitable matching between two sets of points before utilizing the RMSD procedure to fine-tune the final result is by adjusting the suitable parameter sets by ways of searching the underlying parametric space.

Root mean squared deviation

The smallest root mean squared deviation (rmsd) is a least-squares fitting method for two sequences of points [8]. The idea is to align atom vectors of the two given (molecular) structures, and use the common least averaged squared errors as a measurement of differences between these two (paired) sequences. Formally, let $P=\langle p_1,...,p_n\rangle$ and $Q=\langle q_1,...,q_n\rangle$ be two sequences of points. We assume that P is translated so that its centroid $(\frac{1}{n}\sum_{k=1}^{n} p_k)$ is at the origin. We also assume that Q is translated in the same way. For each point or vector x, let $(x)_i(i=1,2,3)$ denote the i-th (X,Y,Z) coordinate value of x,and $\|x\|$ denote the length of x. Let

$$\mathrm{RMSD}(P,Q,R,\mathrm{a})=\sqrt{\frac{1}{n}\sum_{k=1}^{n}\|Rp_k+\mathrm{a}-q_k\|^2},$$

where "R" is a rotation matrix and "a is a translation vector. Then, the $rmsd$ value $d(P,Q)$ between P and Q is defined by $d(P,Q)=\min_{R,\mathrm{a}} d(P,Q,R,\mathrm{a})$. We refer to Martin's ProFit package (standing for protein fitting system) [16] and write a program to calculate the $rmsd$ between C-α atoms of paired protein backbones with C language. Fitting was performed using the McLachlan algorithm [17]

Isometric rotation transformation

According to Euler's rotation theorem [18], any rotation about the origin point can be described by using three angular parameters. The rotation is determined by 3 consecutive rotations with 3 *Euler angles* (α, β, γ). The first rotation is done by the angle α around the *z*-axis, the second is done by the angle β around the *x*-axis, and the third rotation is done by the angle γ around the *z*-axis. See [19] for more detailed discussions about the transformation.

Analog to Euler's setting, we can also consider the point of north-pole $\mathbf{n} = (0, 0, 1)^T$ on the unit sphere. After the rotation, $\boldsymbol{R}$, say **n** is rotated to another point $\mathbf{p} = (x, y, z)^T$; i.e., $\mathbf{p} = \boldsymbol{R}\mathbf{n}$. Let α denote the angle $\angle nOp$. Note that α determines the *z*-coordinate of **p**. To determine *x*-coordinate and *y*-coordinate of **p**, the point is rotated around the *z*-axis for the angle β on the unit sphere. Note that there are infinitely numbers of rotation that transform **n** to **p**. The particular rotation $\boldsymbol{R}$ can be decided by rotating all other points around the vector **p** by the angle γ. It is not hard to verified that, in such a way, any rigid rotation transformation can be parameterized by the three-tuple (α, β, γ). Thus, we call a vector $\mathbf{p} = (x, y, z)^T$ on the surface of the unit sphere a probe. Note that the ***movement*** of each probe is started from the northpole $(0, 0, 1)^T$ to other points in the sphere. The position of **p** is decided by the parameters (α, β), and exact rotation is fixed by the self-rotation angle γ.

As a result, we reduce the problem of finding the good rotation matrix to the new problem of finding a good 3-parameter setting. The rotation matrix is thus characterized by just adjusting the 3 uniformly distributed parameters.

Minimum bipartite matching

We use the minimum bipartite matching algorithm to find the best matching between two sets of points to decide the best matching for the rmsd procedure. We adopted the Munkres [20], [21] algorithm. In order to improve the efficiency of computation, we rewrite the Munkres algorithm, which is implemented on a public available perl module, by C language.

VAST

The structure alignment algorithm of VAST is based on aligning secondary structure elements using an algorithm from the field of graph theory. The output is a neighbors D list. It also contains the complete PDB representative structure comparison structure alignments and a structure superposition tool. The search space for alternative secondary structure elements depends on the length of proteins. All pairs of secondary structure elements (one from each structure) that have the same type are represented as nodes of a graph. This correspondence graph is then searched to find the maximal subgraph such that every node in the subgraph is connected to every other node in the subgraph and is not contained in any larger subgraph with this property. VAST has been used to compare all known PDB domains to each other. The results of this computation are included in NCBI's Molecular Modeling Database at http://www.ncbi.nlm.nih.gov/Structure/VAST/vast.html.

Parametric adjustment with trigonometric series

In the trigonometric series estimation algorithm [22], the three parameters (rotational angles) are assumed to be independent. Three parameters are adjusted one by one, and the estimation function corresponding to the affected *rmsd* values is subsequently better estimated by increasing the curvature degree of the estimated function. The trigonometric series function is described as the following:

$$\begin{aligned} f(\theta) = C_1 &+ C_2 \cos\pi\theta + C_3 \sin\pi\theta \\ &+ C_4 \cos 2\pi\theta + C_5 \sin 2\pi\theta \\ &+ C_6 \cos 3\pi\theta + C_7 \sin 3\pi\theta + \ldots \\ &+ C_{2k} \cos k\pi\theta + C_{2k+1} \sin k\pi\theta. \end{aligned}$$

where $f(\theta)$ denote the corresponding rmsd value with respect of one of the three parameters, (α, β, γ). The *k* usually reflects the numbers of local maximal points in the approximated curve.

we propose algorithms to improve the ***rmsd*** value between a protein structure pair by finding better alignment list. A set of experiments is designed to test the parameters; these adjusted parameters are set to perform the experiments over a thousand of protein pairs, which are uniformly random sampled from the PDB database. As the results shown that our methods improve the alignment computed by the VAST by an averaged improvement ratios about 11 %. The results of figure 1 demonstrate that the method of using 3D Euclidean distance minimum bipartite matching with trigonometric series estimated parametric searching scheme indeed improves existed known system like the VAST.[23]

The initial method method	The average *rmsd* of initial method	The average *rmsd* after trigono. adj.
VAST	1.9572	1.7329 (11.46%)
semi-local-Alig	1.8378	1.7697 (9.58%)
cut1	2.1031	1.7947 (8.30%)
M1	1.9025	1.8047 (7.79%)
cut3	3.4563	1.8526 (5.34%)
M2	2.0612	1.9033 (2.75%)
Main Vector	4.2697	2.5907 (-32.37%)

Figure 1.The rmsd values of trigonometric series adjustment after initial alignment of VAST, Seg-Alig(cut3), Seg-Alig(cut1), Semi-local-Alig, M1, M2 and Main Vector. Notice the improved ratio (in%) comparing to VAST's original rmsd.

since the structure comparison problem, like many scientific computation/simulation problem, is very time-consuming under cases of large structures and large number of paired structures, it is desirable to implement the system under massive parallel machines cluster, e.g., the grid-environment, to increase the throughput of the system.

B. Hadoop MapReduce framework

Hadoop is a software framework for coordinating computing nodes to process distributed data in parallel. Hadoop adopts the map/reduce parallel programming model, to develop parallel computing applications. The standard map/reduce mechanism has been applied in many successful Cloud computing service providers, such as Yahoo, Amazon EC2, IBM, Google and so on. An application developed by Map/Reduce is composed of Map stage and Reduce stage (optionally). Figure 2 illustrates the Map/Reduce framework. Input data will be split into smaller chunks corresponding to the number of Maps. Output of Map stage has the format of <*key, value*> pairs. Output from all Map nodes, <*key, value*> pairs, are classified by key before being distributed to Reduce stage. Reduce stage combines value by key. Output of Reduce stage are <*key, value*> pairs where each key is unique.

Hadoop cluster includes a single master and multiple slave nodes. The master node consists of a jobtracker, tasktracker, namenode, and datanode. A slave node, as computing node, consists of a datanode and tasktracker. The jobtracker is the service within Hadoop that farms out Map/Reduce tasks to specific nodes in the cluster, ideally the nodes that have the data, or at least are in the same rack. A tasktracker is a node in the cluster that accepts tasks; Map, Reduce and Shuffle operations from a jobtracker.

Hadoop Distributed File System (HDFS) is the primary file system used by Hadoop framework. Each input file is split into data blocks that are distributed on datanodes. Hadoop also creates multiple replicas of data blocks and distributes them on datanodes throughout a cluster to enable reliable, extremely rapid computations. The namenode serves as both a directory namespace manager and a node metadata manager for the HDFS. There is a single namenode running in the HDFS architecture.

C. Protein structure analysis on Map/Reduce framwork

Figure 3 illustrates the MapReduce framework for the protein structure analysis. Assume that the number of compute node is *N* and the number of protein pairs lines is *P* in the protein pairs list file, the *P* lines will become *P* map tasks and send to hadoop by streaming program. Each computing node receives a map and then performs the analysis work. When a node completes the map task, the node pass the score to the Reduce and receive next map task to compute unless the total map are finished. Generally, each node will be assigned to deal with *P/N* maps.

Therefore, Reduce will have *P* RSMD scores. The RMSD score is described in the previous section 2.A. The reduce operation writes the protein structure pair analysis scores line by line to the output file on HDFS.

4 Experiments

A. Experimental environment and data soure

All of the experiments were performed on one NFS server and four IBM blade server within our Cloud Computation laboratory. Each server is equipped with two Quad-Core Intel Xeon

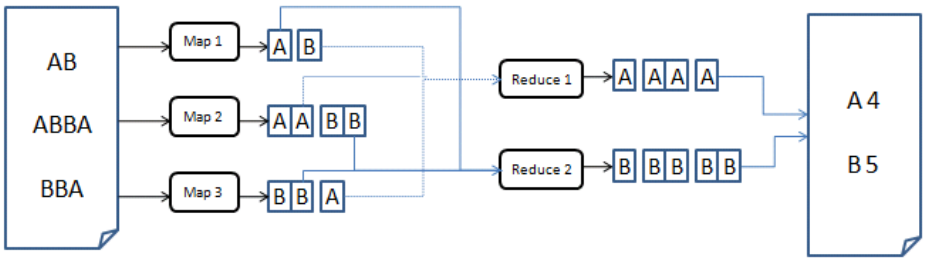

Figure 2. Example of counting words on MapReduce framework. The input data set {(A, B), (A, B, B, A), (B, B, A)} are split into three maps, and each map process its data. Reduce process the data with same key. The output is number of "A" and "B" .

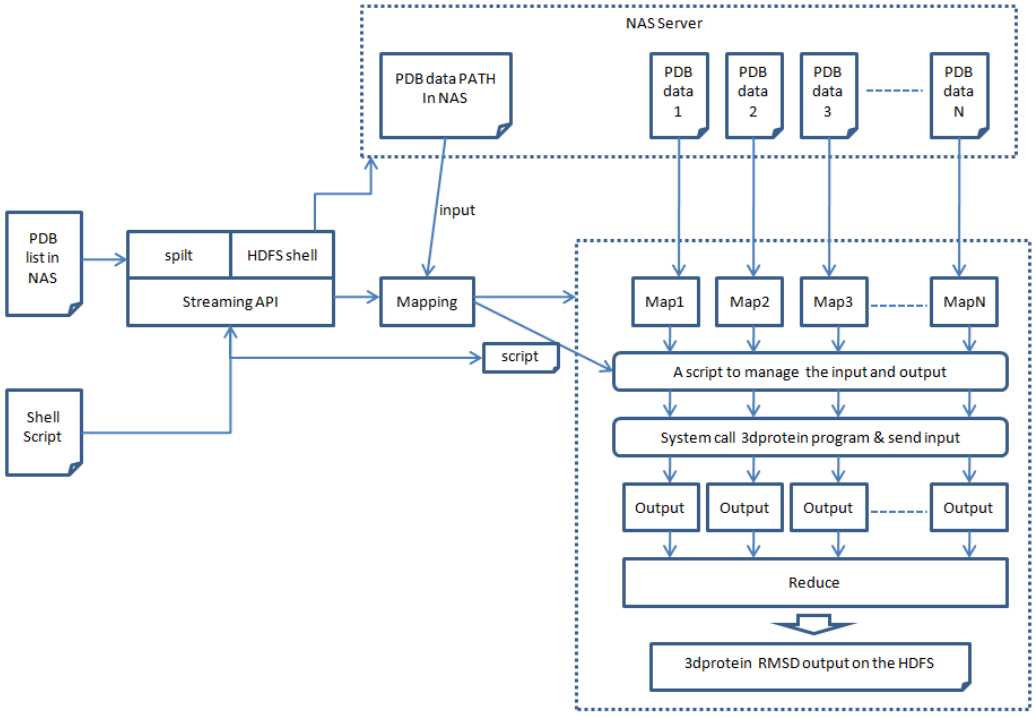

Figure 3. Protein structure analysis use hadoop platform use Map/reduce framework.

2.26GHz CPU, 24G RAM, and 296G disk running under the operation system Ubuntu version 10.4 with KVM and QEMU platform. Under the current system environment, we create 8 virtual machine by KVM, each virtual machine is set one core CPU, 2G RAM, and 30G disk running under the operation system Ubuntu version 10.4 with

Hadoop version 0.2 MapReduce platform, and we control each virtual machine execution processes 1 map operation and 1 reduce operation. The total number of the map/reduce operations are up to 8 respectively.

The Protein structure data sources are gathered from the World Wide Protein Data Bank (http://www.wwpdb.org), which is a PDB format content with 4-latter entry name. This Data Bank maintains this single archive of macromolecular structural data that is freely and publicly available to the global community. We downloaded the protein structure data from the wwPDB's ftp server (ftp://ftp.wwpdb.org/) and the number of protein structure data is 79,180. These data are treated as the input data for our experiments.

B. Experimental result

To assess the performance of the proposed Hadoop MapReduce algorithm, we compare the computational time between various number of structure data and various number of map/reduce operations. Two factors, the number of protein

structure data and the number of protein structure atoms, affect the performance of sequential algorithm and the proposed algorithm. The RMSD scores are calculated with protein ID pairs. Figure 4 and 5 illustrate the comparisons between sequential algorithm and our proposed algorithm under the MapReduce framework. In Fig. 4 and Fig. 5, the maximum atom numbers of protein pairs are between 1-100 and 101-200, respectively.

The experimental result reveals that the computational time is effectively reduced when more map operations are deployed. Comparing to the original sequential algorithm, two, four and eight map operations improve the computation time by factor of two, four and eight times. The computation efficiency can be effectively improved proportional to the number of processors being used.

In Fig. 4 and Fig. 5, it is observed that computational time increases corresponding to number of protein pair and number of protein atom. The computation time with 1-100 atoms is less than that with 101-200 atoms. More number of atom lead to higher computational cost. These experimental results are corresponding to the algorithm analysis in previous section.

C. Web System

We make a web for user who is interesting in our protein structures alignment system at https://sites.google.com/site/pucspsas/.

The user of our web service usually provides two protein PDB IDs, and the service will search protein structure data from wwPDB ftp and perform the desired protein structure comparison using the chosen set of algorithms.

To avoid the time delay, our web service also provides user access keys for user to check the result later on after they submit the desiredquery. User can come back and check the result after operation finished by our local server.

5 Concluding Remarks

Comparion of 3D structures is useful in analyzing and classifying these protein structures in order to discover the structural relationships with protein functions. It possible to establish distant relationships, even between protein families distinct in terms of sequence comparison alone.

Protein structure comparison methods have been mature enough in bioinformatics, but in the implementation of performance above a single computer has been unable to have a better breakthrough.

A good solution is to use parallel computing platform, like Hadoop, and integration into the algorithm, so you can easily enhance the performance of our implementation.

In the sequential version, the algorithm takes a huge time to compute the score between two protein structure, it always spends hours even days. However, in the parallelized-map/reduce version, it's performance can improve the computation time by the factor that the maps/reduces we have. It was greatly speed up the efficiency of this protein structure computation algorithm.

In this paper, we have compared the performances between the sequential version and the parallelized-map/reduce version of our proposed protein structure alignment algorithm. In the future, we will apply more bioinformatics functions to the parallel algorithms to provide more perspectives for biologists to improve the performance of bioinformatic algorithms on for analyzing the increasingly huge bioinformatics data.

Acknowledgment

This research was partially supported by the National Science Council under the Grants NSC-99-2632-E-126-001-MY3

References

[1] M. Gerstein, R. Jansen, T. Johnson, J. Tsai, and W. Krebs.,“Motions in a database framework: from structure to sequence,” *Rigidity Theory and Applications*, pp. 401–442 (ed. M F Thorpe and P M Duxbury, Kluwer Academic/Plenum Publishers), 1999.

[2] N. Echols, D. Milburn, , and M. Gerstein, “Molmovdb: analysis and visualization of conformational change and structural flexibility,” *Nucleic Acids Res.*, vol. 31, p. 478V482, 2003.

[3] S. Dietmann and L. Holm, “Identification of homology in protein structure classification.” *Nature Struct. Biol.*, vol. 8, pp. 953–957, 2001.

[4] J. M. Bujnicki, “Phylogeny of the restriction endonucleaselike superfamily inferred from comparison of protein structures.” *J Mol Evol.*, vol. 50, pp. 38–44, 2000.

[5] M. S. Johnson, M. J. Sutcliffe, and T. L. Blundell, “Molecular anatomy: Phyletic relationships derived from threedimensional structures of proteins.” *J Mol Evol.*, vol. 30, pp. 43–59, 1990.

[6] Y. L. Lin, Y. H. Lin, P. S. Yu, and H. C. Chang, “Randomized algorithms for three dimensional protein structures alignment,” The 6th International Symposium on Computational Biology and Genome Informatics., pp. 122 – 125, 2005.

[7] J. F. Gibrat, T. Madej, and S. H. Bryant, “Surprising similarities in structure comparison,” Curr Opin Struct Biol, vol. 6(3), pp. 377–385, 1996 Jun.

[8] L. Holm and C. Sander, “Touring protein fold space with DALI/FSSP.” Nucleic Acids Res., vol. 26, pp. 316–319, 1998.

[9] I. N. Shindyalov and P. E. Bourne, “Protein structure alignment by incremental combinatorial extension (CE) of the optimal path,” Protein Eng., vol. 11, pp. 739–747, 1998.

[10] Y. L. Lin and S. P. Huang, “Tools and algorithms for refined comparison of protein structures,” in The 6th WSEAS International Conference on Microelectronics, Nanoelectronics, Optoelectronics (MINO ’07), Istanbul, Turkey, 2007.

[11] H. S. Shin, S. P. Huang, and Y. L. Lin, “Parametric searching algorithms with adaptive strategy for three dimensional protein structures alignments,” in National Computer Symposium (NCS’2007), Taichung, Taiwan, 2007, pp. 144–154.

[12] Hadoop - Apache Software Foundation project home page [http://hadoop.apache.org/]

[13] Taylor, R.C., 2010, An overview of the Hadoop/MapReduce/HBase framework and its current applications in bioinformatics. BMC Bioinformatics, 11:S1.

[14] Dean, J., Ghemawat, S., 2010, MapReduce: A Flexible Data Processing Tool. Communications of the ACM, 53, 72-77.

[15] Schatz, M., 2009, Cloudburst: highly sensitive read mapping with MapReduce. Bioinformatics, 25, 1363-1369.

[16] A. C. R. Martin, http://www.bioinf.org.uk /software/profit/.

[17] A. D. McLachlan, “Rapid comparison of protein structures.” Acta Cryst, vol. A38, pp. 871–8783, 1982.

[18] L. Euler, “Formulae generales pro trandlatione quacunque corporum rigidorum,” Novi Acad. Sci. Petrop., vol. 20, pp. 189–207, 1775.

[19] A. Gray, “A treatise on gyrostatics and rotational motion,” MacMillan,London, 1918.

[20] J. Munkres, “Algorithms for the assignment and transportation problems,” Journal of the Society for Industrial and Applied Mathematics, vol. 5, pp. 32–38, 1957.

[21] F. Bourgeois and J. C. Lassalle, “An extension of the munkres algorithm for the assignment problem to rectangular matrices,” in Communications of the ACM, vol. 14. New York, NY: USA, 1971, pp. 802 – 804.

[22] H. S. Shin, Y. L. Lin, and W. D. Jiang., “Protein structures alignment algorithms by parametric searching with trigonometric series,” in Proceedings of the 25th Workshop on Combinatorial Mathematics and Computation Theory, Hsinchu, Taiwan, 2008, pp. 44–54.

[23] Hong-Shin Chen, Yaw-Ling Lin, "Comparisons of Semi-local Alignment Algorithms for Protein Structures" *Proceedings of the 27th Workshop on Combinatorial Mathematics and Computation Theory (CMCT 2010)*, pp V6-254-259, Taichung, Taiwan, April 30-May 1, 2010.

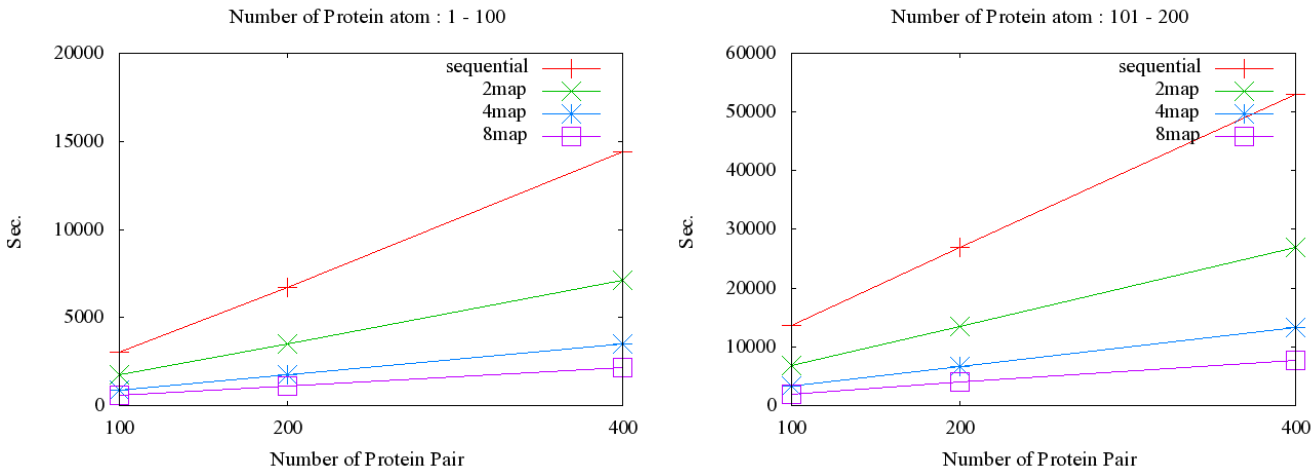

Figure 4. Performance comparison between sequential compute RMSD of protein structure pair and MapReduce compute RMSD of protein structure pair using various number of map. The Number of Protein atom has been split up into two types, 1-100 and 101-200.

100 Pairs				
	1-100		101-200	
Sequential	3037	100%	13587	100%
2map	1745	57.45%	6756	49.72%
4map	892	29.37%	3366	24.77%
8map	586	19.29%	2010	14.79%

200 Pairs				
	1-100		101-200	
Sequential	6679	100%	26914	100%
2map	3510	52.55%	13548	50.33%
4map	1736	25.99%	6719	24.96%
8map	1111	16.63%	3953	14.68%

400 Pairs				
	1-100		101-200	
Sequential	14424	100%	52970	100%
2map	7098	49.20%	26880	50.74%
4map	3494	24.22%	13355	25.21%
8map	2178	15.09%	7698	14.53%

Figure 5. Computing time and the percentage relationship of sequential algorithm and using MapReduce algorithm. The pairs is number of protein structure data pair and be separated form number of protein atom by 1-100 and 101-200